Fayoum Racing Team

Autonomous Technical Team

ROS2 Humble

Comprehensive Documentation & Guide

Prepared by:

Ammar Yasser

President of Fayoum Racing Team

Contents

Overview - Strategic Roadmap: The 7 Phases of the Fayoum Racer **4**

1 Phase 1: The Nervous System (Core Communication) **7**
1.1 The Core Concept: What is a "Robotics Middleware"? 7
1.2 The Philosophy: The Microservices / Nervous System Analogy 8
1.3 The 4 Pillars of ROS 8
1.4 The Computation Graph (Nodes and Executors) 9
1.4.1 Part 1: The Node (The Fundamental Worker) 9
1.4.2 Part 2: Executors (The Engine of the Node) 12
1.4.3 Part 3: The ROS 2 Development Lifecycle (Standard Operating Procedure) 14
1.4.4 Part 4: Inspecting the Computation Graph (CLI Tools) 16

2 Phase 2: Integrated Communication Protocols **19**
2.1 Module 2.1: Topics (Continuous Data Streams) 19
2.1.1 Part 1: Topics (Continuous Data Streams) 19
2.1.2 Part 2: Implementation - The Publisher 20
2.1.3 Part 3: Implementation - The Subscriber 22
2.1.4 Part 4: Inspecting the Data Stream (CLI Tools) 24
2.2 Module 2.2: Services (Synchronous Requests) 27
2.2.1 Part 1: The "Why" & The "How" (Client/Server Architecture) . . 27
2.2.2 Part 2: Implementation - The Service Server 28
2.2.3 Part 3: Implementation - The Service Client (With Async Best Practices) 30
2.2.4 Part 4: Best Practices & Pitfalls (CRITICAL: The Deadlock Trap) 33
2.3 Module 2.3: Actions (Asynchronous, Long-Running Tasks) 34
2.3.1 Part 1: The "Why" & The "How" (Proactive Cancellation) 34
2.3.2 Part 2: Implementation - The Action Client (With Safety Preemption) 34
2.3.3 Part 3: Best Practices & Pitfalls 39
2.3.4 Part 4: Inspecting the Action Graph (CLI Tools) 39

3 Phase 3: The Command Center (Orchestration Tools) **43**
3.1 Module 3.1: Parameters (Dynamic Configuration) 43
3.1.1 Part 1: The "Why" & The "How" (Distributed Parameters) 43
3.1.2 Part 2: Implementation - Dynamic Parameters (The Code) 44
3.1.3 Part 3: Bulk Loading (YAML Configurations) 47

3.1.4 Part 4: Best Practices & Pitfalls (CRITICAL) . . . 47
3.1.5 Part 5: Inspecting and Mutating Parameters (CLI Tools) . . . 48
3.2 Module 3.2: The Launch System . . . 50
3.2.1 Implementation: The Python Launch File . . . 50
3.2.2 Part 2: Launch File Composition (The Master Bringup) . . . 52
3.3 Module 3.3: Debugging and Data Logging . . . 55
3.3.1 Part 1: The "Why" & The "How" (The Black Box & The Visualizer) 55
3.3.2 Part 2: Implementation (CLI Tools) . . . 55
3.3.3 Part 3: Best Practices & Pitfalls (CRITICAL) . . . 57

4 Phase 4: Spatial Awareness & Kinematics 58
4.1 Module 4.1: TF2 (The Transform Framework) . . . 58
4.1.1 Part 1: The "Why" & The "How" (Coordinate Frames & The TF Tree) . . . 58
4.1.2 Part 2: Real-world Autonomous Racing Use Cases . . . 59
4.1.3 Part 3: Implementation - The Static Broadcaster . . . 59
4.1.4 Part 4: Implementation - Dynamic Broadcaster and Listener . . . 60
4.1.5 Part 5: CLI Tools & Debugging (CRITICAL) . . . 63

5 Phase 5: Advanced Node Architecture 65
5.1 Module 5.1: Composition (Component Nodes) . . . 65
5.1.1 Part 1: The "Why" & The "How" (IPC & Zero-Copy) . . . 65
5.1.2 Part 2: Implementation (C++ Component) . . . 66
5.1.3 Part 3: Execution (The Component Container) . . . 68
5.1.4 Part 4: Best Practices & Pitfalls (CRITICAL) . . . 69
5.1.5 Part 5: Automating Components with Launch Files . . . 69
5.2 Module 5.2: Managed Nodes (Lifecycle Nodes) . . . 71
5.2.1 Part 1: The "Why" & The "How" (Deterministic Boot Sequencing) 71
5.2.2 Part 2: Implementation (The State Machine) . . . 71
5.2.3 Part 3: Execution and Transitions (CLI Tools) . . . 73
5.2.4 Part 4: Best Practices & Pitfalls (CRITICAL) . . . 73

6 Phase 6: Hardware & Simulation Integration 74
6.1 Module 6.1: Robot Description (URDF/Xacro) . . . 74
6.1.1 Part 1: The "Why" & The "How" (URDF vs. Xacro) . . . 74
6.1.2 Part 2: Implementation - The XML Structure . . . 75
6.1.3 Part 3: Visualizing the Result . . . 76
6.1.4 Part 4: Best Practices & Pitfalls (CRITICAL) . . . 77
6.2 Module 6.2: `ros2_control` (The Muscle System) . . . 77
6.2.1 Part 1: The "Why" & The "How" (The Ecosystem) . . . 77
6.2.2 Part 2: The URDF Integration . . . 78
6.2.3 Part 3: The YAML Configuration . . . 78
6.2.4 Part 4: Best Practices & Pitfalls (CRITICAL) . . . 79
6.3 Module 6.3: Simulation Context (Gazebo Integration) . . . 79
6.3.1 Part 1: The "Why" & The "How" (The Physics Bridge) . . . 79
6.3.2 Part 2: The Simulation URDF Tweaks (Adding Sensors) . . . 80
6.3.3 Part 3: The Spawn Launch File (Bringing it to Life) . . . 81
6.3.4 Part 4: Best Practices & Pitfalls (CRITICAL) . . . 82

7 Phase 7: The Autonomous Stack **83**
7.1 Module 7.1: Sensor Fusion & State Estimation (EKF) 83
7.1.1 Part 1: The ”Why” & The ”How” (The Kalman Filter) 83
7.1.2 Part 2: The `robot_localization` Configuration 84
7.1.3 Part 3: Implementation (The Launch) 85
7.1.4 Part 4: Best Practices & Pitfalls (CRITICAL) 85
7.2 Module 7.2: Perception & Mapping (SLAM) 86
7.2.1 Part 1: The ”Why” & The ”How” (SLAM vs. EKF) 86
7.2.2 Part 2: Real-world Autonomous Racing Use Cases 86
7.2.3 Part 3: The `slam_toolbox` Configuration 86
7.2.4 Part 4: Implementation (Launch & RViz) 87
7.2.5 Part 5: Best Practices & Pitfalls (CRITICAL) 88
7.3 Module 7.3: Nav2 (The Navigation Framework) 88
7.3.1 Part 1: The ”Why” & The ”How” (The Modular Navigator) 88
7.3.2 Part 2: Real-world Autonomous Racing Use Cases 88
7.3.3 Part 3: The Nav2 Configuration (Ackermann Specific) 89
7.3.4 Part 4: Costmap Management (Global vs. Local) 89
7.3.5 Part 5: Best Practices & Pitfalls (CRITICAL) 90

Strategic Roadmap: The 7 Phases of the Fayoum Racer

1. Phase 1: The Nervous System (Core Communication)

> *"To build the fundamental 'Nervous System' of the racer. This phase isn't just about code; it's about establishing how every sensor and motor acts as a single, synchronized organism using the ROS 2 Computation Graph."*

What the team will master:

- Transforming a single script into a distributed network of **Nodes**.
- Mastering the **Executor** logic to ensure no sensor data is lost during high-speed processing.
- Adhering to the **ROS 2 SOP** to maintain professional-grade code stability.

—

2. Phase 2: Integrated Communication Protocols

> *"To master the language of robotics. We are defining the protocols for how the car 'talks'—from the non-stop stream of LiDAR data to the critical mission commands that must never fail."*

What the team will master:

- Creating robust **Publishers/Subscribers** for continuous telemetry and sensor feeds.
- Designing **Services** for instant, reliable hardware triggers (like resetting an encoder).
- Implementing **Actions** for complex maneuvers with proactive cancellation safety.

—

3. Phase 3: The Command Center (Orchestration & Tools)

> *"To take full control of the vehicle's behavior. This is where we move from running parts to orchestrating a complete racing machine that can be tuned, launched, and diagnosed with professional precision."*

What the team will master:

- Tuning performance in real-time using **Dynamic Parameters** without restarting the system.
- Automating the entire autonomous stack deployment with a single **Master Launch File**.
- Utilizing **Rosbag2** as our 'Flight Data Recorder' and **RViz2** as our visual eyes.

—

4. Phase 4: Spatial Awareness & Kinematics

> *"To give the racer a sense of space. We are building the 'Digital Geometry' of the car, ensuring that every sensor knows exactly where it is relative to the track and the chassis, down to the last millimeter."*

What the team will master:

- Managing complex **Coordinate Frames** (Map, Odom, Base_link) to prevent localization errors.
- Solving real-world racing problems like sensor displacement and wheel-to-lidar calibration.
- Mastering **TF2 Listeners and Broadcasters** to maintain a perfect 3D transform tree.

—

5. Phase 5: Advanced Node Architecture

> *"To optimize the racer for extreme performance. We are moving beyond standard nodes to build a high-speed, deterministic system that can handle 4K vision and high-freq LiDAR without breaking a sweat."*

What the team will master:

- Using **Composition** and **Zero-Copy** communication to eliminate CPU bottlenecks.
- Enforcing a **Deterministic Boot Sequence** using **Lifecycle Nodes** for safety.
- Orchestrating complex shared-memory containers for vision-heavy perception tasks.

—

6. Phase 6: Hardware & Simulation Integration

> *"To bridge the gap between code and reality. We are creating the 'Digital Twin' of the Fayoum Racer, allowing us to test dangerous maneuvers in Gazebo before ever touching the physical track."*

What the team will master:

- Describing the car's physics and geometry using modular **URDF/Xacro** files.
- Implementing **ros2_control** to handle the 'Muscles' (Ackermann steering and motor torque).
- Mastering the **Gazebo Physics Bridge** to simulate real-world sensor feeds.

—

7. Phase 7: The Autonomous Stack

> *"To awaken the Master Driver. This is the ultimate goal: fusing sensors, mapping the track, and planning the optimal racing line to win the race autonomously."*

What the team will master:

- Implementing the **Extended Kalman Filter (EKF)** to find the "Filtered Truth."
- Building high-resolution track maps using **SLAM** while localizing at high speeds.
- Orchestrating the **Nav2 Framework** to compute the fastest path and avoid obstacles.

CHAPTER 1

Phase 1: The Nervous System (Core Communication)

1.1 The Core Concept: What is a "Robotics Middleware"?

The "Why": Why can't we just write a standalone C++ program?

Imagine you want to build an autonomous race car. Your first instinct might be to open an IDE and write a massive C++ program with a giant `while(true)` loop: read the LiDAR, process the camera frames, calculate the path, and send PWM signals to the steering servo.

This monolithic approach fails instantly in the real world for three reasons:

- **Concurrency and Bottlenecks:** Processing a 4K camera stream takes significant compute time. If your `while` loop is stuck waiting for an image frame, your car isn't sending braking commands. You will crash.
- **The Single Point of Failure:** In a monolithic script, a segmentation fault caused by a buggy camera driver crashes the entire program. The steering and brakes freeze because the camera failed.
- **Reinventing the Wheel:** You would have to write custom drivers for every single piece of hardware from scratch, parsing raw bytes over USB or Ethernet.

The "How": The Middleware Solution

ROS is a "Middleware." It sits between the operating system (like Linux) and your specific robotics software. It provides a standardized framework that allows you to break that giant monolithic C++ program into dozens of small, independent, specialized programs (called **Nodes**) that run concurrently and talk to each other.

1.2 The Philosophy: The Microservices / Nervous System Analogy

Think of ROS like a biological nervous system or a modern cloud microservices architecture.

In the human body, your eyes don't know how your legs work. Your eyes simply capture light, convert it into standard neural signals, and publish them to the brain. The brain subscribes to those signals, makes a decision, and publishes movement commands to the legs.

ROS does exactly this for robots:

- **A Camera Node** (the eye) only knows how to talk to a specific physical camera. It translates raw USB data into a standard ROS `Image` message and broadcasts it.
- **A Perception Node** (the visual cortex) subscribes to that `Image`, detects track boundaries, and broadcasts a `TrackBounds` message.
- **A Control Node** (the motor cortex) subscribes to the bounds and calculates steering angles.

If you swap your camera for a completely different brand, the Perception and Control nodes don't care. They don't need to be rewritten. They just keep waiting for that standard `Image` message.

1.3 The 4 Pillars of ROS

To make this distributed nervous system work, ROS provides four fundamental pillars:

- **1. Hardware Abstraction (The Drivers):**
  What it is: ROS provides pre-written, open-source packages that act as translators between raw hardware and the ROS ecosystem.
  Real-world Use Case: When you buy a Hokuyo LiDAR or an Ouster 3D LiDAR, you don't read the manufacturer's manual to decode byte streams. You just run the ROS driver provided by the manufacturer or community, and instantly, standard `LaserScan` or `PointCloud2` messages start flowing into your system.
- **2. Message Passing (The Plumbing):**
  What it is: The standardized communication protocols (which we will cover via DDS) that allow a Python script running your AI to seamlessly pass data to a highly optimized C++ node handling motor control, as if they were the same program.
  Real-world Use Case: A complex autonomous vehicle might have an object detection algorithm written rapidly in Python, streaming bounding box coordinates to a path-planning algorithm written in Modern C++ 17 for maximum performance. ROS handles the serialization and network transit between the two languages invisibly.
- **3. Package Management (The Ecosystem):**
  What it is: A standardized way to organize, build, and share code.
  Real-world Use Case: You don't need to write a complex navigation algorithm from scratch. Because ROS uses standardized packages, you can download the `Nav2`

package, configure it for your vehicle's kinematics, and instantly have state-of-the-art autonomous navigation.

- **4. Tools (The Laboratory):**
  What it is: A suite of powerful GUI tools for visualization, debugging, and simulation.
  Real-world Use Case: Before putting a physical vehicle on a track—where a software bug means destroyed hardware—you use Gazebo to simulate the physics of the car and the sensors. You use RViz2 to visually inspect what the robot's "brain" is seeing in real-time (e.g., overlaying the LiDAR point cloud onto a 3D grid).

Best Practices & Common Pitfalls

- **Pitfall – The Monolithic Node:** Beginners often adopt ROS but still write "God Nodes"—a single ROS node that handles camera processing, path planning, and control all at once. This defeats the entire purpose of the framework.
- **Best Practice – Single Responsibility Principle:** A node should do exactly one thing. One node reads the sensor. One node filters the data. One node plans the path. This makes debugging easy: if the path is bad, you check the output of the filtering node. If the filter output is good, you know the bug is strictly isolated inside the planner.

1.4 The Computation Graph (Nodes and Executors)

1.4.1 Part 1: The Node (The Fundamental Worker)

The "Why"

As we discussed, a monolithic program is a liability. We need a way to encapsulate specific hardware interactions or algorithms into isolated, independent processes. The "Node" is the atomic unit of computation in ROS 2. It is the entity that connects to the DDS middleware, registers itself on the network, and handles its own specific job.

The "How" (Architecture Level)

Under the hood, a ROS 2 Node is not a standalone magical entity; it is simply a C++ or Python class that inherits from the ROS Client Library base classes (`rclcpp::Node` or `rclpy.node.Node`). When you instantiate this class, it communicates with the underlying DDS implementation to announce its existence to the rest of the computation graph.

Use Cases

- **The Sensor Driver:** A node specifically dedicated to reading wheel encoders on an autonomous race car to calculate raw speed.
- **The Safety Watchdog:** A lightweight node that constantly monitors system diagnostics and triggers an emergency stop if the main planner crashes.

Implementation: Writing Your First Node

Industry standard dictates that ROS 2 code must always be Object-Oriented. We never write procedural scripts in the global scope. We will write a simple "Telemetry Node" that boots up and logs its status at a fixed frequency.

C++ Implementation (Modern C++ 14/17)

Create this in `src/telemetry_node.cpp`:

```cpp
#include <chrono>               // Standard library for time/duration
#include <memory>               // Standard library for smart pointers (Modern C++)
#include "rclcpp/rclcpp.hpp"    // The core ROS 2 C++ client library
using namespace std::chrono_literals; // Allows us to write '500 ms' instead of verbose chrono calls

// 1. Inherit from the base rclcpp::Node class
class TelemetryNode : public rclcpp::Node
{
public:
    // 2. The Constructor: Name the node "telemetry_monitor" on the ROS graph
    TelemetryNode() : Node("telemetry_monitor"), count_(0)
    {
        // 3. Create a timer that fires every 500 milliseconds.
        // std::bind hooks the timer to our member function 'timer_callback'
        timer_ = this->create_wall_timer(
            500ms, std::bind(&TelemetryNode::timer_callback, this));

        // Log a startup message to the console
        RCLCPP_INFO(this->get_logger(), "Telemetry Node Initialized. Systems nominal.");
    }
private:
    // 4. The Callback: This function executes every time the timer fires
    void timer_callback()
    {
        RCLCPP_INFO(this->get_logger(), "Publishing telemetry packet #%d", count_++);
    }

    // Member variables
    rclcpp::TimerBase::SharedPtr timer_; // Smart pointer managing the timer's lifecycle
    size_t count_;                       // Simple counter
};

int main(int argc, char * argv[]) {
```

```cpp
    // A. Initialize the ROS 2 communications network
    rclcpp::init(argc, argv);

    // B. Instantiate our node using a shared pointer (Modern C++ best practice)
    auto node = std::make_shared<TelemetryNode>();

    // C. Hand the node over to the Executor to spin (block and process callbacks)
    rclcpp::spin(node);

    // D. Clean shutdown of the ROS 2 network when the node is killed (Ctrl+C)
    rclcpp::shutdown();
    return 0;
}
```

Python Implementation

Create this in `telemetry_node.py`:

```python
import rclpy                                # The core ROS 2 Python client library
from rclpy.node import Node                 # The base Node class

# 1. Inherit from the base rclpy.node.Node class
class TelemetryNode(Node):

    def __init__(self):
        # 2. The Constructor: Name the node "telemetry_monitor" on the ROS graph
        super().__init__('telemetry_monitor')
        self.count_ = 0

        # 3. Create a timer that fires every 0.5 seconds, triggering 'timer_callback'
        self.timer_ = self.create_timer(0.5, self.timer_callback)

        # Log a startup message to the console
        self.get_logger().info('Telemetry Node Initialized. Systems nominal.')

    # 4. The Callback: This function executes every time the timer fires
    def timer_callback(self):
        self.get_logger().info(f'Publishing telemetry packet #{self.count_}')
        self.count_ += 1

def main(args=None):
    # A. Initialize the ROS 2 communications network
    rclpy.init(args=args)
```

```
26
27      # B. Instantiate our node
28      node = TelemetryNode()
29
30      # C. Hand the node over to the Executor to spin (block and
            process callbacks)
31      rclpy.spin(node)
32
33      # D. Clean shutdown of the ROS 2 network when the node is
            killed (Ctrl+C)
34      node.destroy_node()
35      rclpy.shutdown()
36
37  if __name__ == '__main__':
38      main()
```

1.4.2 Part 2: Executors (The Engine of the Node)

The "Why"
Look at the `main` function in both examples. You created a node, but then you called `spin()`. Why? Because a Node by itself is just a passive data structure. It does nothing until an "Executor" gives it a thread to run on. If a network packet arrives from the LiDAR, something has to actively wake up, read the packet, and trigger your callback function.

The "How" (Architecture Level)
The Executor is the actual engine of ROS 2 computation. It manages one or more threads and a queue of callbacks (timers, incoming messages, service requests).

`rclcpp::spin(node)` creates a Single-Threaded Executor by default. It takes the calling thread (the main thread) and locks it into an infinite loop, checking the DDS queue and executing callbacks one by one, in order.

Use Cases

- **Single-Threaded Executor:** Perfect for our Telemetry Node above. It only has one timer, so one thread is plenty.
- **Multi-Threaded Executor:** Crucial for complex nodes like an autonomous path planner. If the planner receives a massive PointCloud message that takes 100ms to process, you do not want the node to ignore an emergency stop command during those 100ms. A multi-threaded executor allows the emergency command to be processed on thread #2 while thread #1 is busy with the PointCloud.

Best Practices & Common Pitfalls

The Deadliest Sin: Blocking the Executor Thread.

- **Pitfall:** In a Single-Threaded Executor, if you put `sleep(5)` or a heavy `while(true)` mathematical calculation inside your `timer_callback`, the en-

tire node freezes. No other timers will fire. No incoming messages will be read. Your vehicle will fly completely blind for 5 seconds.
- **Best Practice:** Callbacks must be lightning-fast. They should read data, update state variables, and exit immediately. If you have heavy computation, push it to a separate background thread or use a Multi-Threaded Executor with mutually exclusive Callback Groups.

Object-Oriented Design (OOD):
- **Pitfall:** Declaring global variables to share data between different ROS callbacks.
- **Best Practice:** As demonstrated in the code, encapsulate everything within the class state (e.g., `self.count_`). This guarantees thread safety (when configured properly) and allows you to instantiate multiple instances of the same node type in a single process later on.

1.4.3 Part 3: The ROS 2 Development Lifecycle (Standard Operating Procedure)

The "Why"
Without a strict SOP, engineers will compile code in the wrong directories, overwrite environment variables, or run outdated binaries. This lifecycle enforces the "Underlay/Overlay" architecture we discussed earlier, ensuring that your custom code seamlessly sits on top of the base ROS 2 framework without corrupting it.

The "How" (The Rhythm)
This is the chronological loop you will execute hundreds of times a week as a ROS 2 developer.

Step 1: Environment Setup (The Underlay)
Before you can use any ROS 2 tools (`ros2`, `colcon`), your terminal needs to know they exist. You must source the base ROS 2 installation.

Terminal
```
# This injects ROS 2 Humble commands and libraries into your current
terminal.
source /opt/ros/humble/setup.bash
```

Step 2: Workspace Creation
You isolate your project from the rest of your Linux system by creating a dedicated workspace with a `src` directory.

Terminal
```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src
```

Step 3: Package Generation
You generate the boilerplate architecture for your specific module.

Terminal
```
# For a C++ Node:
ros2 pkg create my_cpp_pkg --build-type ament_cmake --dependencies
rclcpp

# OR For a Python Node:
ros2 pkg create my_python_pkg --build-type ament_python
--dependencies rclpy
```

Step 4: Implementation (Code & Build Registration)
This is where you write the actual logic (the `telemetry_node` code we wrote in Part 1).
Crucial Step: You must register your node so the build system knows it exists.

- **C++:** You edit `CMakeLists.txt` to add the executable (`add_executable`) and

install rules.

- **Python:** You edit `setup.py` and add your node to the `console_scripts` entry points.

Step 5: Compilation (colcon)
You return to the root of your workspace to compile. You **never** compile from inside the `src` folder.

Terminal

```
cd ~/ros2_ws

# The --symlink-install flag is mandatory for modern development.
# It creates symbolic links to Python scripts and config files.
# This means if you edit a Python file, you do NOT have to rebuild!
colcon build --symlink-install
```

Step 6: Environment Registration (The Overlay) - CRITICAL
`colcon` just built your code and put the binaries in the `install` folder. However, your terminal still does not know they exist. You must source the newly created overlay.

Terminal

```
# This tells your terminal: "Look in here for my custom packages."
source install/setup.bash
```

Step 7: Execution
Now, and only now, can you run the node.

Terminal

```
ros2 run my_cpp_pkg telemetry_node
```

Step 8: Verification
To verify it works, you must open a new terminal. Because it is a new terminal, it is completely blank. You must repeat the sourcing steps before using the CLI tools.

Terminal

```
# In Terminal 2:
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash

# Interrogate the graph
ros2 node list
ros2 node info /telemetry_monitor
```

Best Practices & Common Pitfalls

- **Pitfall – The "Ghost Code" Phenomenon:** You edit a C++ file, type

`ros2 run`, and nothing changes. You edit it again, still nothing. Why? Because you forgot to run `colcon build`. C++ must always be recompiled.

- **Pitfall – The "Cross-Contaminated Workspace":** If you have multiple workspaces (e.g., `navigation_ws` and `perception_ws`), never source them both in the same terminal while building. Sourcing an overlay before running `colcon build` can accidentally link your new code to the wrong dependencies.
- **Best Practice – The `.bashrc` Automation:** Typing `source /opt/ros/humble/setup.bash` in every single new terminal is exhausting. Engineers automate this by adding it to their `~/.bashrc` file so it runs automatically every time a terminal opens. However, you generally do **not** automate sourcing your custom overlay (`install/setup.bash`) to prevent cross-contamination during active development.

1.4.4 Part 4: Inspecting the Computation Graph (CLI Tools)

The "Why"
ROS 2 runs as a distributed system. You might have 50 different nodes running simultaneously across three different computers on the same network. We need specialized Command Line Interface (CLI) tools to launch these nodes, verify their existence, and dissect their internal connections in real-time without stopping the system.

The "How" (Architecture Level)
Under the hood, these CLI tools are essentially temporary, hidden ROS 2 nodes themselves. When you type a command like `ros2 node list`, the CLI spins up a temporary node, queries the DDS discovery protocol for active participants, prints the results to your terminal, and immediately shuts itself down.

1. Executing the Code: `ros2 run`
The "Why": You cannot simply run a ROS 2 executable by typing `./telemetry_node`. The executable needs the ROS 2 environment variables, workspace paths, and middleware configurations to successfully join the network.
The "How": `ros2 run` searches your currently sourced workspaces (both the `/opt/ros/humble` underlay and your custom `ros2_ws` overlay) to locate the package and its registered executables.

Use Cases

- Booting up an individual hardware driver (e.g., spinning up just the LiDAR node to test if the laser is spinning).
- Testing a newly compiled C++ or Python node in isolation before integrating it into the full software stack.

Implementation

Terminal

```
# Syntax: ros2 run <package_name> <executable_name>

# Example (Assuming we built the C++ package from Part 1):
ros2 run my_cpp_pkg telemetry_node

# Expected Terminal Output:
# [INFO] [1708214532.123456] [telemetry_monitor]: Telemetry Node
Initialized. Systems nominal.
# [INFO] [1708214532.623456] [telemetry_monitor]: Publishing
telemetry packet #0
# [INFO] [1708214533.123456] [telemetry_monitor]: Publishing
telemetry packet #1
```

2. Network Verification: `ros2 node list`
The "Why": Just because a node hasn't crashed in the terminal doesn't mean it successfully connected to the ROS 2 network. We need a way to confirm that the middleware sees the node.
The "How": This command asks the DDS middleware to return the namespace and node name of every active participant it currently tracks.

Use Cases

- **Sanity Checks:** "I just started the camera driver, but the perception algorithm is failing. Is the camera node even alive on the network?"
- **Network Partition Debugging:** If you run this on your laptop and don't see the nodes running on the robot's onboard computer, you instantly know you have a Wi-Fi or DDS configuration issue, not a code bug.

Implementation
Open a new terminal (ensure you have sourced your workspace) while your `telemetry_node` is running:

Terminal

```
ros2 node list

# Expected Terminal Output:
# /telemetry_monitor
```

3. Deep Introspection: `ros2 node info`
The "Why": Knowing a node is alive is only step one. We need to know its interface: What data is it broadcasting? What data is it listening for? What services does it offer? If node A isn't talking to node B, `node info` is your diagnostic scalpel.
The "How": This command queries the specific node directly over the ROS 2 graph to map out its active Publishers, Subscribers, Service Servers, Service Clients, and Action Servers.

Use Cases

- **Debugging Communication Drops:** Verifying if the motor control node is actually subscribed to the `/cmd_vel` steering topic, or if there is a typo in the code.
- **Exploring Third-Party Code:** When you download a community package (like an IMU driver) and want to quickly see what topics it outputs without digging through its source code.

Implementation

Terminal

```
# Syntax: ros2 node info <node_name>
ros2 node info /telemetry_monitor

# Expected Terminal Output:
# /telemetry_monitor
#   Subscribers:
#     /parameter_events: rcl_interfaces/msg/ParameterEvent
#   Publishers:
#     /parameter_events: rcl_interfaces/msg/ParameterEvent
#     /rosout: rcl_interfaces/msg/Log
#   Service Servers:
#     /telemetry_monitor/describe_parameters:
rcl_interfaces/srv/DescribeParameters
#     /telemetry_monitor/get_parameters: rcl_interfaces/srv/GetParamete
#     ...
```

Note: Even though we didn't explicitly write any publishers or subscribers in our C++ code yet, ROS 2 automatically attaches default parameter and logging interfaces (`/rosout`) to every node.

Best Practices & Common Pitfalls

- **Pitfall – Relying on `ros2 run` for Production:** Beginners try to start an entire robot by opening 15 terminals and typing `ros2 run` in each one. This is unmanageable.
- **Best Practice:** Use `ros2 run` strictly for isolated development and debugging. When deploying the full system, you will use `ros2 launch` (which we will cover in Module 3.2) to bring up dozens of nodes deterministically with a single command.
- **Pitfall – Name Collisions:** If you use `ros2 run` to start the exact same node twice, the second node will boot up with the identical name (`/telemetry_monitor`). This causes massive routing confusion in the DDS middleware. Always ensure node names are unique, or use namespaces (e.g., `/robot1/telemetry_monitor`).

CHAPTER 2

Phase 2: Integrated Communication Protocols

2.1 Module 2.1: Topics (Continuous Data Streams)

2.1.1 Part 1: Topics (Continuous Data Streams)

The "Why"
Imagine an autonomous car driving at 30 km/h. The LiDAR is spinning at 10Hz, generating thousands of data points per second. The motor controller needs constant updates on the target velocity.

You cannot use a "Request/Response" system for this. The motor controller shouldn't have to constantly ask, "What is my speed now? How about now?" Topics exist to solve the need for **continuous, decoupled, unidirectional data streams**.

The "How" (Architecture Level)
Under the hood, Topics use the DDS Publish/Subscribe pattern.

- **Decoupled & Anonymous:** A node publishing to a topic has *no idea* who is listening. A node subscribing to a topic has *no idea* who is sending the data. They only care about the Topic Name (e.g., `/vehicle/speed`) and the Message Type (e.g., `std_msgs/Float32`).
- **Many-to-Many:** One LiDAR node can publish to `/scan`. Five different nodes (Mapping, Obstacle Avoidance, Visualizer, Data Logger, Safety Watchdog) can all subscribe to `/scan` simultaneously without bottlenecking the publisher.

Real-world Autonomous Racing Use Cases

- **Publishers:** Wheel encoders publishing the current physical velocity of the car; a camera driver continuously publishing raw image frames; an IMU publishing linear acceleration and angular velocity.
- **Subscribers:** The dashboard UI listening to the velocity to display a speedometer; the perception algorithm subscribing to the camera frames to detect track bounds;